

Lab2 实验报告

一、思考题

1. CPU发出地址

(1) 编写的C程序中，指针变量存储的地址为**虚拟地址** (2) MIPS 汇编程序中 lw 和 sw 指令使用的地址也是**虚拟地址**

理解：只要你在写用户程序，你看到的所有地址都是虚拟地址。物理地址你永远见不到。

2. 链表的宏实现与不同操作的开销

思考题1：从可重用性的角度，阐述用宏来实现链表的好处

用宏实现链表具有以下可重用性优势：

- 类型安全与泛型**：通过宏模板可以为不同的结构体类型生成类型特定的链表操作，避免了使用**void***指针带来的类型安全问题，同时实现了类似泛型的功能。
 - 类比C++中的 **template**
- 代码高度复用**：同一套链表操作宏可以用于任何需要链表功能的结构体，无需为每种数据类型重复编写链表操作代码，大大减少了代码冗余。
- 编译时展开优化**：宏在预处理阶段展开为内联代码，避免了函数调用的开销，同时编译器可以进行更好的优化。
- 内存效率**：链表节点直接嵌入到结构体中，不需要额外的包装节点，减少了内存分配次数和内存碎片。
- 可移植性**：一套经过充分测试的链表宏可以在不同项目和平台间重用。

思考题2：查看实验环境中的queue.h，了解其中单向链表与循环链表的实现，比较它们与本实验中使用的双向链表，分析三者插入与删除操作上的性能差异

通过分析实验环境中的**queue.h**文件，三种链表的实现和性能对比如下：

1. 单向链表 (SLIST - Singly Linked List)

```
#define SLIST_ENTRY(type) struct { struct type *sle_next; }
```

- 结构特点**：每个节点只有一个指向后继节点的指针
- 插入操作**：
 - 头部插入：O(1) - 只需更新头指针和新节点的**next**
 - 尾部插入：O(n) - 需要遍历到链表尾部
 - 任意位置插入：如果知道前驱节点则为O(1)，否则需要遍历找到插入位置
- 删除操作**：
 - 头部删除：O(1) - 只需更新头指针

- 任意节点删除：O(n) - 需要遍历找到前驱节点才能更新指针
- **空间开销**：每个节点1个指针
- **适用场景**：主要用于LIFO（后进先出）栈或只需要头部操作的队列

2. 双向链表（LIST - Doubly Linked List，本实验使用）

```
#define LIST_ENTRY(type) struct { struct type *le_next; struct type **le_prev; }
```

- **结构特点**：每个节点有指向后继和前驱的双重指针（`le_prev`是指向前驱节点`next`指针地址的二级指针）
- **插入操作**：
 - 任意位置插入：O(1) - 只要有节点引用，就可以直接更新前后节点的指针
 - 头部/尾部插入：O(1) - 通过头结构可以直接访问
- **删除操作**：
 - 任意节点删除：O(1) - 通过节点的`le_prev`可以直接修改前驱节点的`next`指针
- **空间开销**：每个节点2个指针（`le_next`和`le_prev`，其中`le_prev`是二级指针）
- **优势**：任意位置插入删除都是常数时间，无需遍历

3. 循环链表（CIRCLEQ - Circular Queue）

```
#define CIRCLEQ_ENTRY(type) struct { struct type *cqe_next; struct type *cqe_prev; }
```

- **结构特点**：双向链表的变体，尾节点的`next`指向头节点，头节点的`prev`指向尾节点，形成闭环
- **插入删除性能**：与双向链表相同，任意位置操作都是O(1)
- **特殊处理**：
 - 空链表检测：头节点的`next`和`prev`都指向自身
 - 遍历需要特殊终止条件（遇到头节点时停止）
- **优势**：
 - 可以从任意节点开始遍历整个链表
 - 实现循环缓冲区等特殊数据结构更方便
 - 头尾操作更对称

性能对比总结：

操作类型	单向链表(SLIST)	双向链表(LIST)	循环链表(CIRCLEQ)
头部插入	O(1)	O(1)	O(1)
尾部插入	O(n)	O(1)	O(1)
任意位置插入	O(n)找到位置 + O(1)插入	O(1)	O(1)
头部删除	O(1)	O(1)	O(1)
尾部删除	O(n)找到前驱	O(1)	O(1)
任意节点删除	O(n)找到前驱	O(1)	O(1)

操作类型	单向链表(SLIST)	双向链表(LIST)	循环链表(CIRCLEQ)
空间开销	1指针/节点	2指针/节点	2指针/节点
遍历方向	单向	双向	双向循环

实验中的双向链表实现通过`le_prev`二级指针的巧妙设计，使得删除操作无需知道前驱节点的完整信息，只需常数时间即可完成，这是相比传统双向链表设计的优化之处。

3. Page_list的结构与宏的使用

选项C

```
struct Page_list{
    struct {
        struct{
            struct Page* le_next;
            struct Page** le_prev;
        } pp_link;
        u_short pp_ref;
    }* lh_first;
};
```

因此 我如果想取出`le_prev`

```
struct Page_list* p
p -> pp_link.le_prev
```

我如果想要遍历空闲页表

```
for(struct Page *p = page_free_list->lh_first; p != NULL; p = p->pp_link.le_next);
```

其实就是`LIST_FOREACH(struct Page *p, page_free_list, pp_link)`

4. ASID与TLB

问题1：从虚拟内存和多进程操作系统的实现角度，阐述ASID的必要性

在虚拟内存和多进程系统中，ASID（Address Space Identifier，地址空间标识符）具有以下必要性：

- 区分不同进程的地址空间：**每个进程拥有独立的虚拟地址空间，不同进程可能使用相同的虚拟页号(VPN)映射到完全不同的物理页面。ASID作为地址空间的标识，使得TLB可以通过`<VPN, ASID>`的组合键唯一标识一个页表项，从而正确区分不同进程的地址映射。
- 避免进程切换时完整刷新TLB：**如果没有ASID，每次进程切换时，操作系统必须将整个TLB无效化（刷新），因为TLB中缓存的旧进程页表项对新进程来说都是无效的。这会导致新进程运行时大量TLB缺失，需

要频繁陷入内核重填，性能严重下降。ASID允许TLB中同时保留多个进程的页表项，进程切换时只需要切换当前ASID，不需要刷新整个TLB，大大提高了进程切换速度和TLB利用率。

3. **提高TLB访问效率**：ASID使得TLB可以直接在硬件层面完成地址空间区分，不需要软件通过标记无效、清空等方式处理多进程地址空间隔离，保持了TLB硬件查找的高效性。
4. **全局共享页节省空间**：对于内核空间等所有进程共享的代码和数据，可以通过设置G（Global）位忽略ASID检查，使得这些共享页面的TLB表项只需要存储一份，节省TLB空间。

问题2：结合ASID段的位数，说明4Kc中可容纳不同的地址空间的最大数量

MIPS 4Kc中ASID字段为8位。

因此，理论上最多可以表示 $2^8 = 256$ 个不同的地址空间。

二、难点分析与实验体会

1. 核心难点（如命令使用/逻辑实现等）

内存分布

kuseg - 最大，需要经过TLB/页表翻译 kseg0 - 不需要经过页表翻译，直接减去0x80000000得到物理地址

- 所以想直接访问物理地址，只需要加上0x80000000 转化为逻辑地址的kseg0段，再由CPU操作访问即可。【转化为虚拟地址的方法KADDR
- `struct Page*`（统一都在可用内存前，集中分布页表的信息）、物理页面首地址、kseg0段的va虚拟地址 三者转换
 - `pa2page(*pte) page2pa(pp)` 负责物理页面首地址与页表结构体的映射
- 当我要对地址具体内容修改时，我应当对kseg0段的va虚拟地址进行修改，而非物理地址！！

访问指定页表的内存 $(pp - pages) * PAGE_SIZE + 0x80000000$ 考试时只需要使用 `page2kva(pp)`；即可

内核的内存分布

freemem之前包含 .data .text .bss 以及所有struct page 结构体，每一个结构体指向物理内存的一页（双向映射）

- 这些内核的部分不能被分配，所以应当标记为已使用

两级页表

页表项的结构 划分地址结构：以 | 10 页目录号 | 10 页号 | 12 偏移 | 总共32位地址为例，其PTE结构是

1. 物理地址：

- 低12位：读取时抹零，一定为0，因为**页对齐**；所以作为 权限位(perm)
- 高20位：
- 不能直接读，要转化为kseg0(+0x80000000)

2. PTE_V: 有效位

3. PTE_D: 是否可写
4. PTE_G: 使得所有进程的虚拟地址指向同一个物理地址

二级页表查询

TLB

- 硬件机制
- 访存时 不需要陷入内核、执行查找、返回，而是先查找TLB
- 但是 TLB 需要与页表“同步” **TLB 表项结构** Index : | 32 key | 32(Page1) | 32(Page2) |
- 映射 (VA, Asid) -> 类似于页表项(PA, PERM)
- Asid: 每一个进程标识位 key
 - 高19位 (-1) : 存储VA的高19位 (20-1) == VPN 减一位 Page
 - 页表项 >> 6

奇偶页机制 两个32位分别为奇数页、偶数页

如何与TLB交互

一些新的 mips 指令 RTL语言 ****tlbr: **** Index -> (EntryHi, EntryLo0, EntryLo1) ****tlbp: **** EntryHi(TLB key) -> Index if va not exist then Index < 0

****tlbwi: **** 事先将data -> EntryHi, EntryLo0, EntryLo1,

(EntryHi, EntryLo0, EntryLo1) -> Index

TLB 重填 注意 写回TLB，只能由CPU操作，因此只能通过tlbwi/tlbwr汇编指令写入，因此我需要通过C程序，将参数设定好a0 a1 a2，传给汇编。

2. 实验收获/技巧总结

1. 链表操作宏定义化。

2. 双向链表的小巧思

前驱指针使用双重指针

```
#define LIST_ENTRY(type)
struct {
    struct type *le_next; /* next element*/
    struct type **le_prev; /* address of previous next element */ //解引用得到
    上一页的le_next指针，这样的话就不需要判断指针为空，（因为指针变量一定有地址）
}
```

3. 多参数返回

使用指针传参，

```
int page_alloc(struct Page **new) {
    struct Page *pp;
    ...
    *new = pp;
    return 0;
}
```

4. 转换

3. 实验注意事项

1. 链表的操作赋值
2. 课下**一定对着答案检查**，每一道题都有可能测试通过，但出现新的问题!!!

三、参考资料

1. 实验指导文档

- [MIPS32 指令集参考](#)
- lab 2 考前复习: [lab 2 pre-Exam.pdf](Instruction/lab 2 pre-Exam.pdf)
- [MIPS32 User's Manual](#)

2. 课程讲义

- 刘学长的视频 <https://www.bilibili.com/video/BV1VPDuBuEeb>

3. 参考书籍

- Randal E. Bryant, **深入理解计算机系统** — 虚拟存储器章节
- William Stallings, **操作系统：精髓与设计原理** — 内存管理章节